

Universidade Federal de Pernambuco

Centro de Informática - CIn

Sistema de Gestão de Doação e Transplante de Órgãos



Relatório de MongoDB

Banco de dados - 2026.1

Repositório Github: <https://github.com/philipsantiagoo/mongodb-project>

Arthur Dias de Oliveira - ado

Felipe Augusto Lira Dias - fald

Gabriel André Souza Fernandes da Cunha - gasfc

Luiz Philip Santiago da Silva - lpss

Vinicius Arraes de Almeida - vaa

Felipe Mateus Pereira da Silva - fmps

Sumário

Documentação Projeto MongoDB.....	4
1. Tema e Visão Geral do Projeto.....	4
2. Descrição Detalhada da Aplicação (Modelagem).....	4
Coleções do Banco:.....	4
3. Scripts e Implementação da Checklist (MongoDB).....	4
3.1 Setup Inicial e Povoamento.....	5
3.2 Atualizações, Modificações e Remoções (CRUD).....	5
3.3 Consultas Básicas.....	5
3.4 Agregações e Consultas Avançadas.....	6

Documentação Projeto MongoDB

1. Tema e Visão Geral do Projeto

O projeto baseia-se na implementação de um **Sistema de Gestão de Doação e Transplante de Órgãos**. O sistema gerencia o fluxo completo entre doadores (vivos e post-mortem) e pacientes na fila de espera, passando pelo registro das instalações hospitalares, cadastro de equipes médicas habilitadas e o acompanhamento detalhado dos transplantes realizados e seus índices de compatibilidade.

A aplicação foi inteiramente modelada utilizando os paradigmas de bancos de dados orientados a documentos NoSQL, sendo desenvolvida utilizando a linguagem específica do MongoDB.

2. Descrição Detalhada da Aplicação (Modelagem)

O armazenamento dos dados é centralizado no banco lógico *orgaos_db* e foi construído em cinco coleções interligadas, desenhadas para extrair o máximo de recursos não-relacionais, como arrays e subdocumentos.

Coleções do Banco:

- **doadores:** Armazena dados de pessoas cadastradas para doação. Cada documento contém os dados pessoais do indivíduo, indicadores biomédicos vitais (como tipo sanguíneo e os antígenos de histocompatibilidade - HLA), tipagem (vivo ou post-mortem), e um array de subdocumentos para os *orgaos_disponiveis* (contendo detalhes como condição e lateralidade do órgão).
- **receptores:** Registra os pacientes que aguardam uma oportunidade de transplante. Agrupa dados clínicos e gerenciais essenciais, incluindo a priorização (*urgencia* de 1 a 5), o status do indivíduo na fila e um array de subdocumentos (*historico_tentativas*) que traça tentativas falhas por rejeição.
- **hospitais:** Centraliza as instalações de saúde capacitadas. Grava a infraestrutura hospitalar com dados geográficos de endereço, contato, contagem de leitos e o mapeamento de habilitações cirúrgicas.
- **medicos:** Cadastra os profissionais responsáveis pelas cirurgias de transplante. Armazena as especialidades do médico em array, o número de procedimentos realizados, status de disponibilidade da agenda e uma referência direta de qual hospital é a sua origem de atendimento.
- **transplantes:** A coleção transacional que registra o procedimento operatório. Ela une as referências (*ObjectId*) do doador, receptor, médico e hospital. Inclui campos analíticos do procedimento, como resultados pós-cirúrgicos ("sucesso", "rejeição" etc.), duração cirúrgica em minutos e um subdocumento de *compatibilidade* cruzando o pareamento genético e sorológico da operação.

3. Scripts e Implementação da Checklist (MongoDB)

Para cobrir integralmente as requisições de seleção, filtros e agregações exigidas pela especificação, a aplicação divide a lógica de negócio em uma estrutura de scripts modulares. Todos os 31 itens da checklist obrigatória foram contemplados nas rotinas do banco.

Abaixo, os scripts estão descritos de acordo com a regra de negócio aplicada:

3.1 Setup Inicial e Povoamento

- **1. USE:** O comando `use("orgaos_db");` é executado na inicialização de todos os scripts para o direcionamento do banco em uso.
- Na configuração inicial, são montados os índices simples nos campos de busca constante, bem como índices compostos e de busca textual (`TEXT` e `SEARCH`).
- O script `02_insert.js` realiza o cadastro de 50 doadores, 50 receptores, 10 hospitais, 50 médicos e 50 transplantes gerados randomicamente via Javascript para permitir consultas realistas através da instrução de inserção múltipla.

3.2 Atualizações, Modificações e Remoções (CRUD)

Operações contempladas no script `03_crud.js`:

- **21. SET / 25. UPDATEONE e UPDATEMANY:** O operador `$set` foi utilizado de forma dupla. Primeiramente associado a uma atualização única (`updateOne`) redefinindo a quantidade de leitos e o email da diretoria de um hospital de nome compatível. Em seguida, acoplado ao `updateMany` para ativar a flag de disponibilidade de todos os médicos que detêm a especialidade de "cirurgia_transplante".
- **Remoção:** Utilizou-se o método `deleteOne` para fazer uma limpeza operacional, removendo o primeiro registro cirúrgico constando status de "cancelado".
- **26. SAVE (UPDATEONE / INSERTONE):** Considerando as melhores práticas de versões recentes do MongoDB que depreciaram a função `.save()`, o recurso foi implementado através de rotinas de `insertOne` para criação pura de novos receptores, em conjunto com operações de `updateOne` recebendo o modificador `{ upsert: true }`, criando um registro médico apenas caso o CRM referenciado não seja encontrado.
- **27. RENAMECOLLECTION:** O comando nativo é acionado via script administrativo para alterar o nome da coleção `transplantes` para uma versão de arquivamento chamada "historico_transplantes".

3.3 Consultas Básicas

Operações contempladas no script *04_queries_basic.js*:

- **2. FIND / 19. PRETTY**: Rotina convencional de busca com o método de formatação tabular `.pretty()` aplicada em massa (ex: selecionando doadores com tipo sanguíneo de urgência "O+").
- **30. FINDONE**: Recupera precisamente a via única do primeiro objeto documental de receptor sinalizado pelo status aguardando doação.
- **14. SORT / 15. LIMIT**: Em conjunto dinâmico, ordena a lista dos receptores combinando urgência decrescente com a data de entrada fila, aplicando limitação nas devoluções textuais das queries focando no Top 5.
- **24. FILTER (Simples)**: Aplicação da filtragem de documento mapeando um objeto dentro do subdocumento verificando `orgaos_disponiveis.orgao` igual a "figado" e sua disponibilidade positiva.
- **6. PROJECT**: A exclusão dos dados confidenciais nos retornos do shell limitou explicitamente as exibições a apenas `nome` e `tipo` num mapeamento focado sobre os campos públicos utilizando as flags numéricas (`{nome: 1, _id: 0}`).
- **13. EXISTS**: A função de checagem condicional retorna ocorrências em que a cirurgia teve um preenchimento lógico válido na chave `resultado`.
- **7. GTE**: Filtro comparador aritmético isola para triagem pacientes cadastrados cujo fator da avaliação de urgência médica estivesse superior ou equivalente a 4.
- **3. SIZE**: Verificação na estrutura de arrays de comprimento validando os indivíduos (doadores) onde a lista do número de órgãos equivale estritamente a 2.
- **16. \$WHERE**: Adiciona processamento via declaração Javascript nas procuras de banco validando um comparador do vetor via `this.historico_tentativas.length > 2`.
- **10. COUNT (COUNTDOCUMENTS)**: Levantamento estatístico unificado contabilizando o volume bruto gerado apenas para entidades marcadas internamente com doador "post-mortem".
- **20. ALL**: Verifica a existência totalitária das tags "A2" e "B7" concomitantemente nas propriedades biomoleculares da base do `hla` cadastrado no array do doador.
- **22. TEXT / 23. SEARCH**: Aplicação combinada e dependente aos índices pré-fabricados aciona uma verificação no texto pesquisando ocorrências sintáticas de "nefrologia" nas listagens médicas ativas.

3.4 Agregações e Consultas Avançadas

Operações contempladas no script *05_queries_advanced.js*:

- **4. AGGREGATE / 5. MATCH:** Estrutura fundamental construída nas procuras avançadas de banco orquestrando fluxos e reduzindo o universo de análises dos resultados que geraram flag de "sucesso".
- **8. GROUP / 9. SUM:** A instrução totalizadora aciona o acúmulo da variável de contagem (**\$sum**) unindo a indexação aos registros isolados do campo **\$orgao** nos registros de operação.
- **11. MAX / 12. AVG:** Inseridos nativamente nos cálculos do bloco agrupador do pipeline, os identificadores processam em massa as estimativas mensuráveis verificando a minutagem extrema de demora e efetuando a medição paramétrica central média do procedimento **\$duracao_cirurgia_min**.
- **24. FILTER (Avançado):** Utilizado diretamente durante a conversão do agregado para reconstruir apenas os subdocumentos das amostras em que o avaliador booleano **disponivel** se mantinha **true** sem expor registros fechados ou vencidos.
- **28. COND:** Empregou blocos lógicos estruturados **if-then-else** para criar chaves dinâmicas na saída de processamento textual das urgências numéricas variando de "BAIXO" e "MODERADO" até "CRÍTICO".
- **29. LOOKUP:** Promove as montagens complexas equivalentes ao JOIN cruzando a identidade de ligação das coletas estrangeiras (**foreignField: "_id"**) a partir da transação para trazer o nome real do usuário tanto das listagens de hospitais como dos receptores cruzados com doadores.
- **31. ADDTOSET:** Desmonta a array no estágio de **\$unwind** e recria na conversão acumuladora salvaguardando repetições desnecessárias para montar um pacote puramente focado em listagens de órgãos absolutos disponíveis por estrutura cirúrgica hospitalar.
- **17. MAPREDUCE:** Declara a dupla lógica contida (**emit**) somando variáveis mapeadas para criar relatórios gerenciais das tendências dos perfis variados nos tipos sanguíneos registrados em banco.
- **18. FUNCTION:** Permite processar scripts puros sob a rotulagem **\$function**, executando cruzamentos condicionais e complexos iterando em tempo real comparando a compatibilidade entre listas de HLA por código base do objeto doador.